

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

Course Code: CSE 219 Course Title: Signals and Linear Systems

Time: 25 minutes

Marks: 16 + 4 bonus

Student Name: Md. Ashrafur Rahman Khan

Student ID: 1905005

Answer all of Questions 1–2. Question 3 is for bonus marks. Answer concisely.

1. Consider the sequence

$$x[n] = \{2, 0, 1, -1\}, \quad 0 \leq n < 4$$

You want to compute its 4-point DFT using the **radix-2 DIF FFT**.

[8]

(a) Perform the first butterfly stage and write the intermediate values.

Solution: In radix-2 DIF FFT, the first stage combines the pairs $(x[0], x[2])$ and $(x[1], x[3])$.

From the first pair:

$$a_0 = x[0] + x[2] = 2 + 1 = 3,$$

$$a_2 = (x[0] - x[2])W_4^0 = (2 - 1) \cdot 1 = 1.$$

From the second pair:

$$a_1 = x[1] + x[3] = 0 + (-1) = -1,$$

$$a_3 = (x[1] - x[3])W_4^1 = (0 - (-1))W_4^1 = 1 \cdot (-j) = -j.$$

So after the first stage, the intermediate values are

$$\{a_0, a_1, a_2, a_3\} = \{3, -1, 1, -j\}.$$

(b) Complete the second stage and write the outputs in the order produced directly by the DIF algorithm.

Solution: The second stage applies 2-point butterflies separately to (a_0, a_1) and (a_2, a_3) :

$$y_0 = a_0 + a_1 = 3 + (-1) = 2,$$

$$y_1 = a_0 - a_1 = 3 - (-1) = 4,$$

$$y_2 = a_2 + a_3 = 1 + (-j) = 1 - j,$$

$$y_3 = a_2 - a_3 = 1 - (-j) = 1 + j.$$

Therefore, the outputs produced directly by the DIF flow are

$$\{y_0, y_1, y_2, y_3\} = \{2, 4, 1 - j, 1 + j\}.$$

(c) Rearrange the result from part (b) to obtain the DFT in natural order, i.e. $X[0], X[1], X[2], X[3]$.

Solution: For radix-2 DIF FFT, the outputs come out in bit-reversed order. For $N = 4$, the bit-reversal mapping is

$$0 \rightarrow 0, \quad 1 \rightarrow 2, \quad 2 \rightarrow 1, \quad 3 \rightarrow 3.$$

So the algorithm-order output

$$\{2, 4, 1 - j, 1 + j\}$$

corresponds to

$$\{X[0], X[2], X[1], X[3]\}.$$

Hence the DFT in natural order is

$$X[0] = 2, \quad X[1] = 1 - j, \quad X[2] = 4, \quad X[3] = 1 + j.$$

(d) Briefly state where bit-reversal appears in radix-2 DIT FFT and where it appears in radix-2 DIF FFT.

Solution: In radix-2 DIT FFT, bit-reversal appears at the **input** side: the algorithm usually expects the inputs in bit-reversed order and produces outputs in natural order.

In radix-2 DIF FFT, bit-reversal appears at the **output** side: the inputs are in natural order, and the algorithm produces outputs in bit-reversed order.

2. In an audio-processing application, a discrete-time signal $x[n]$ of length L is passed through an FIR filter $h[n]$ of length M to produce

$$y[n] = (x * h)[n].$$

For large L and M , you want to compute this efficiently.

[8]

(a) What is the length of the output sequence $y[n]$? What is the time complexity of direct time-domain computation?

Solution: If $x[n]$ has length L and $h[n]$ has length M , then their linear convolution has length

$$L + M - 1.$$

A direct implementation computes, for each output sample, a sum over filter taps. This leads to time complexity

$$O(LM).$$

- (b) If you compute DFTs of insufficient length, multiply them, and then take an IDFT, what kind of convolution do you obtain instead of linear convolution?

Solution: You obtain **circular convolution**, not linear convolution.

In the audio context, this causes wrap-around: samples from the tail of the filtered signal fold back and corrupt the beginning of the output. So the result is not the correct FIR-filtered signal.

- (c) Propose an FFT-based algorithm to compute the correct filtered output.

Solution: A correct FFT-based method is:

```
1 function FastFiltering(x, h):  
2     Choose N  
3     Zero-pad x and h to length N  
4     X = FFT(x_padded)  
5     H = FFT(h_padded)  
6     Y = X .* H           // pointwise multiplication  
7     y_padded = IFFT(Y)  
8     Return the first L+M-1 samples
```

This works because multiplication in the frequency domain corresponds to convolution in the time domain, and the zero-padding prevents wrap-around from corrupting the result.

- (d) If the transform length is N , what condition must N satisfy? What is the resulting asymptotic complexity?

Solution: To make circular convolution equal linear convolution, the transform length must satisfy

$$N \geq L + M - 1.$$

In practice, we often choose the next convenient FFT length, often a power of 2.

The algorithm uses two FFTs, one pointwise multiplication, and one inverse FFT. Therefore its asymptotic complexity is

$$O(N \log N).$$

This is much better than the direct $O(LM)$ method for large signals.

3. Thank you for attending the classes and participating in this CT.

[4]

Solution: You're welcome!